```
"""
Priority Engine module
Smart Study Planner and Exam Readiness Analyzer
Course: 2026CSAE100 - Computational Thinking and Programming

Functions:
    get_days_left()      - days between today and an exam date
    calculate_priority() - priority score (0 to 1) for one topic
    rank_topics()        - scores every topic and sorts highest to lowest

Priority formula:
    urgency      = max(0, (30 - days_left) / 30)   # 1.0 on exam day, 0 if 30+ days
away
    weakness     = (6 - confidence) / 5            # confidence rated 1 to 5
    difficulty_n = difficulty / 5                  # difficulty rated 1 to 5
    priority     = 0.40*urgency + 0.35*weakness + 0.25*difficulty_n
"""

from datetime import date

# ---- Constants (easy to explain and change) ----
W_URGENCY = 0.40
W_WEAKNESS = 0.35
W_DIFFICULTY = 0.25
URGENCY_WINDOW = 30  # days; exams further away than this have urgency 0


def get_days_left(exam_date, today=None):
    """Return the number of days from today to exam_date (negative if passed)."""
    if today is None:
        today = date.today()
    return (exam_date - today).days


def calculate_priority(days_left, confidence, difficulty):
    """
    Return the priority score (0 to 1) for one topic.

    days_left  : int, days until the exam (0 = exam today)
    confidence : int 1 to 5 (5 = very confident)
    difficulty : int 1 to 5 (5 = very hard)
    """
    urgency = max(0, (URGENCY_WINDOW - days_left) / URGENCY_WINDOW)
    weakness = (6 - confidence) / 5
    difficulty_n = difficulty / 5
    priority = (W_URGENCY * urgency
```

```
                + W_WEAKNESS * weakness
                + W_DIFFICULTY * difficulty_n)
    return round(priority, 2)


def rank_topics(topics, exam_dates, today=None):
    """
    Score every topic and rank from highest to lowest priority.

    topics     : list of dicts with keys
                 subject, topic, difficulty, confidence, hours_needed
    exam_dates : dict mapping subject -> datetime.date
    today      : optional date (defaults to today; useful for testing)

    Returns a list of dicts (same keys plus 'days_left' and 'priority'),
    sorted by priority in descending order. Topics whose exam date has
    already passed (days_left < 0) or has no exam date are skipped.
    """
    ranked = []
    for t in topics:
        subject = t["subject"]
        if subject not in exam_dates:
            continue  # no exam date stored for this subject

        days_left = get_days_left(exam_dates[subject], today)
        if days_left < 0:
            continue  # exam already over

        score = calculate_priority(days_left,
                                   int(t["confidence"]),
                                   int(t["difficulty"]))
        item = dict(t)
        item["days_left"] = days_left
        item["priority"] = score
        ranked.append(item)

    # Highest priority first; on a tie, the sooner exam comes first
    ranked.sort(key=lambda x: (-x["priority"], x["days_left"]))
    return ranked


def print_ranking(ranked):
    """Display the ranked list as a table (used by the 'Rank' menu option)."""
    if not ranked:
        print("No upcoming topics to rank.")
        return
    print(f"\n{'Rank':<5}{'Subject':<12}{'Topic':<18}{'Days':<6}"
          f"{'Diff':<6}{'Conf':<6}{'Priority':<8}")
    print("-" * 61)
    for i, t in enumerate(ranked, start=1):
```

```
        print(f"{i:<5}{t['subject']:<12}{t['topic']:<18}{t['days_left']:<6}"
              f"{t['difficulty']:<6}{t['confidence']:<6}{t['priority']:<8.2f}")


# ---- Quick self-test: run this file directly ----
if __name__ == "__main__":
    from datetime import timedelta

    today = date.today()
    sample_topics = [
        {"subject": "Physics", "topic": "Electrostatics", "difficulty": 5,
         "confidence": 2, "hours_needed": 6},
        {"subject": "Maths", "topic": "Matrices", "difficulty": 3,
         "confidence": 4, "hours_needed": 4},
        {"subject": "Chemistry", "topic": "Bonding", "difficulty": 2,
         "confidence": 5, "hours_needed": 3},
        {"subject": "English", "topic": "Essay", "difficulty": 2,
         "confidence": 3, "hours_needed": 2},  # exam already passed
    ]
    sample_exams = {
        "Physics": today + timedelta(days=6),
        "Maths": today + timedelta(days=15),
        "Chemistry": today + timedelta(days=40),
        "English": today - timedelta(days=2),
    }

    # Test cases from the report
    assert calculate_priority(6, 2, 5) == 0.85    # Physics example
    assert calculate_priority(15, 4, 3) == 0.49   # Maths example
    assert calculate_priority(0, 1, 5) == 1.0     # boundary: exam today, worst case
    print("All tests passed.")

    print_ranking(rank_topics(sample_topics, sample_exams, today))
```